PukSquash Developer Manual

Court Booking Creation

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

This functionality reserves one court for one fixed time slot on one date. OPEN creates a session another user can join. LOCKED reserves the cell without a join action. PRIVATE_COACH also references a coach who has published availability for that date and slot.

The user selects an available cell in BookCourtPage and completes BookingDialog. bookingService posts the selected identifiers and booking type. Laravel resolves the creator from the Firebase identity, validates the request, checks the court and time, checks conflicting user participation and court occupancy, and validates any selected coach. It creates the booking and then the creator's Player 1 record.

1 1 Files and Responsibilities

resources/js/components/BookingDialog.vue — booking type and coach selection

resources/js/features/booking/bookingService.js — createBooking and availableCoaches

app/Http/Controllers/Api/BookingController.php — store

app/Repositories/BookingRepository.php and BookingPlayerRepository.php — mutations

dataconnect/example/mutations.gql — CreateBooking and AddBookingPlayer

1 2 API Contract

POST /api/bookings accepts court_id, time_slot_id, booking_date, booking_type and optional coach_id. PRIVATE_COACH requires coach_id. Success is HTTP 201 with {"id":"<booking ID>"}. The caller cannot choose created_by_user_id.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Validation and identity resolution

app/Http/Controllers/Api/BookingController.php | source line 31 onward; blank and comment lines omitted

```
public function store(Request $request): JsonResponse
{
    $request->validate([
        'court_id'     => 'required|string',
        'time_slot_id' => 'required|string',
        'booking_date' => 'required|date_format:Y-m-d',
        'booking_type' => 'required|in:OPEN,LOCKED,PRIVATE_COACH',
        'coach_id'     => 'nullable|string',
    ]);
    $firebaseUid = $request->attributes->get('firebase_uid');
    $user = $this->users->findByFirebaseUid($firebaseUid);
```

Explanation

The controller validates the external REST fields before doing the domain checks. All three booking types share the endpoint. The Firebase UID comes from middleware and resolves the application user, ensuring the reservation belongs to the authenticated caller.

Physical Code Explanation

1 The request validation fixes the date format and limits booking_type to OPEN, LOCKED or PRIVATE_COACH.

2 coach_id is nullable at initial validation because it is irrelevant to normal bookings. A later branch requires it for PRIVATE_COACH.

3 The controller reads firebase_uid from request attributes and queries the application user. It does not trust a user ID submitted by the client.

The remaining checks return 422 for an inactive/missing court, a past or current slot, or overlapping user participation; SLOT_TAKEN and COACH_UNAVAILABLE use 409. They must run again even when the browser offered the cell as available.

3 Creating the relational booking

dataconnect/example/mutations.gql | source line 159 onward; blank and comment lines omitted

```
mutation CreateBooking(
  $courtId: UUID!
  $timeSlotId: UUID!
  $createdById: UUID!
  $coachId: UUID
  $bookingDate: Date!
  $bookingType: String!
  $bookedAt: Timestamp!
)
@auth(level: NO_ACCESS) {
  booking_insert(
    data: {
      courtId: $courtId
      timeSlotId: $timeSlotId
      createdById: $createdById
      coachId: $coachId
      bookingDate: $bookingDate
      bookingType: $bookingType
      status: "CONFIRMED"
      bookedAt: $bookedAt
    }
  )
}
```

Explanation

The repository passes the controller's camelCase variables to CreateBooking. The mutation inserts a CONFIRMED Booking linked to the Court, TimeSlot, creator and optional coach. The response key is used for a separate AddBookingPlayer call with player position 1.

Physical Code Explanation

1 UUID variables identify the related entities, while bookingDate is a GraphQL Date and bookedAt is a Timestamp.

2 createdById is derived from the verified user. The stored status is explicitly CONFIRMED.

3 NO_ACCESS keeps ordinary client operation execution closed; Laravel uses privileged server access.

4 The controller accepts several insertion-key response shapes, extracts id, then adds Player 1. A missing key becomes HTTP 500.

This excerpt is the Firebase baseline implementation. The two inserts are separate external calls, and the preceding slot check is also separate. Do not describe them as a single transaction or a demonstrated concurrency guarantee.

4 Verification and Maintenance

Verification checklist

1 Create OPEN and LOCKED bookings in different free cells. Verify the Booking row and Player 1 relation, then refresh the grid.

2 Attempt the same court/date/slot again: expect 409 SLOT_TAKEN when the existing booking is observed.

3 Attempt a second booking at the same date/slot on another court: expect 422 USER_ALREADY_BOOKED.

4 Create PRIVATE_COACH without a coach and with an unavailable coach: expect 422 and 409 respectively.

5 Run concurrent requests and simulate a failure between Booking and Player 1 creation. Inspect for duplicate cells or orphan bookings; these are required integrity tests, not proven passing behaviour.

Implementation boundaries

The supplied specification requires normal booking limits, consecutive-booking limits and transactional conflict protection. The baseline store method does not fully implement those limits or one atomic reservation operation.

The schema contains comments describing composite booking and player uniqueness but does not declare all those constraints. Implement and verify a transaction-safe operation and database constraints before relying on concurrent booking safety.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).